

DEPARTMENT OF COMPUTER SCIENCE FALL 2025

**CS4110 - High Performance
Computing with GPUs**

**Deliverable 3
Report**

By:

Fatima Sohail [23i-0633]

Hareem Zia [23f-0842]

HPC-A

Below are our measured times for V2 and V3 on 3 different datasets

V2:

Dataset	Frames	Features	Resolution	Total tracking time(CPU) (s)	Total tracking time(GPU) (s)	Speed up
Provided	10	150	240p	0.08	0.037553	2.13x
Traffic	550	150	480p	12.658749	5.521432	2.29x
City	202	150	1080p	36.306262	8.622948	4.21x

Dataset	Frames	Features	trackFeaturesKernel(ms)	convolveImageHoriz(ms)	convolveImageVert(ms)
Provided	10	150	3.420076	0.154786	0.160417
Traffic	550	150	81.575009	16.160542	16.29183
City	202	150	50.468669	30.330623	31.58694

V3:

Dataset	Frames	Features	Resolution	Total tracking time(CPU)(s)	Total tracking time(GPU)(s)	Speed up
Provided	10	150	240p	0.049	0.005074	9.65x
Traffic	550	150	480p	6.512728	0.757819	8.59x
City	202	150	1080p	19.030554	0.762501	24.95x

trackFeaturesKernel (ms)	convolveHorizShared (ms)	convolveVertShared (ms)	subsampleKernel (ms)	convertToFloatKernel (ms)
1.424549	0.163425	0.167392	0.013889	0.015744
62.435209	17.383582	18.293941	0.853367	2.194822
24.190748	33.08951	35.174974	1.078143	3.922008

Launch configuration

In Version 2, the kernel launch configurations were kept relatively generic, with a fixed block size of 16×16 (256 threads) for most image-processing kernels and one-dimensional launches for feature tracking. This approach ensured correctness but did not fully exploit the GPU’s architectural parallelism or streaming multiprocessor (SM) resources. The lack of fine-tuning in grid and block dimensions led to lower occupancy in compute-intensive kernels like **trackFeaturesKernel**, where thread-level resource usage limited active warps per SM.

In Version 3, the launch configurations were refined based on the profiling results and GPU architecture (RTX 3080, SM 8.6). Each kernel was launched with grid and block dimensions according to its computational pattern and memory access behavior. The **convolution** kernels were optimized with 2D block structures having 256 threads (16×16 in horizontal and 32×8 in vertical) aligned to image dimensions and shared memory tiles, improving coalesced memory access and reducing redundant global reads. For the newly added lightweight kernels (**convertToFloatKernel** and **subsampleKernel**), large grids with high thread counts (256 each) were launched to ensure full coverage of the image domain and maximize parallel throughput. The **trackFeaturesKernel** maintained a balanced distribution of 1 thread

per feature, optimizing warp utilization within available registers despite being computationally heavy. Overall, the refined launch configurations in V3 allowed for better parallel scaling, higher occupancy, and improved performance across all datasets.

Occupancy

In V2 (naive implementation), the GPU occupancy across kernels varied between moderate to high, depending on register usage and memory access patterns. The **convolveHorizKernel** and **convolveVertKernel** achieved near-full theoretical occupancy (~90–100% with all 256 threads in use since they are lightweight and primarily memory-bound. However, the **trackFeaturesKernel** exhibited slightly lower occupancy (~60–70%) due to high register usage and complex per-feature computations, which limited the number of active warps per SM on the RTX 3080 (Compute Capability 8.6). As a result, while the convolution stages were efficiently parallelized, overall GPU utilization was constrained by the tracking stage's register pressure and synchronization overheads.

In V3, the overall occupancy of the application improved significantly. The addition of two lightweight kernels, **convertToFloatKernel** and **subsampleKernel**, achieved near-maximum theoretical occupancy (~95–100%), with all 256 threads being utilized, and because of their simple arithmetic operations, low register demand, and coalesced global memory access. These kernels allowed image preprocessing and pyramid generation to remain fully on the GPU, removing CPU–GPU stalls and maintaining steady GPU engagement. The convolution kernels retained their high occupancy (~90–100%) with shared memory optimizations, and benefited from reduced memory latency due to better data locality.

Communication Optimisations

The communication overhead has been significantly optimised in V3 in comparison to V2. Our results reveal a significant reduction in data transfer overhead through batching and pipeline restructuring. In V2, excessive host-device transfers dominated execution time, with transfers consuming **~2579.14 ms (HtoD)** and **~2523.749ms (DtoH)** for the city dataset, indicating massive back and forth communication and bottlenecked performance. The V3 implementation eliminated that by keeping the image pyramids, gradient data, and intermediate convolution results on GPU between operations, reducing the communication overhead to just **~13.91 (DtoH)** and **~48.70 (HtoD)** which is a **~81.5%** reduction in data movement. This was achieved by restructuring the convolution and pyramid pipelines to maintain data locality, and batching all feature data which executes in parallel for each feature in a single kernel launch. The optimization shifted the computational profile from transfer-dominated to kernel-execution dominated, while achieving **6x** better overall speedup for this dataset.

Memory Optimisations

The V1 (CPU only) implementation relied solely on RAM with sequential memory accesses leading to significant cache misses. V2 introduced GPU memory but used it inefficiently. In V2, there were frequent **cudaMalloc** operations (**~893.24ms**), indicating poor memory reuse with excessive **HtoD (~838.08ms)** and **DtoH(~595.44ms)** transfers (on the traffic dataset) rather than persisting on GPU.

On the other hand, V3 has exploited constant memory for convolution kernels to broadcast access to all threads, shared memory tiling in convolution kernels (**convolveVertShared** and **convolveHorizShared**), eliminating redundant global memory accesses by caching image tiles with halos, and unified device memory persistence kept entire pyramids and intermediate results on GPU. Moreover, shared memory has been utilized for feature windows (temporary gradient/difference data) to eliminate redundant accesses to global memory. This hierarchy reduced the **cudaMalloc** overhead from **~893.24ms** to **~423.93ms (2.11%)**, while also contributing to reduction in communication overhead by **~54.61%**.